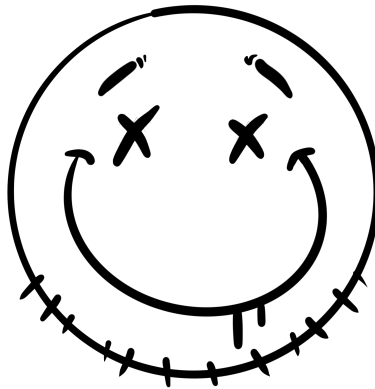




Lab 08: IDS Configuration

Keith Schoen

CSEC3755-001

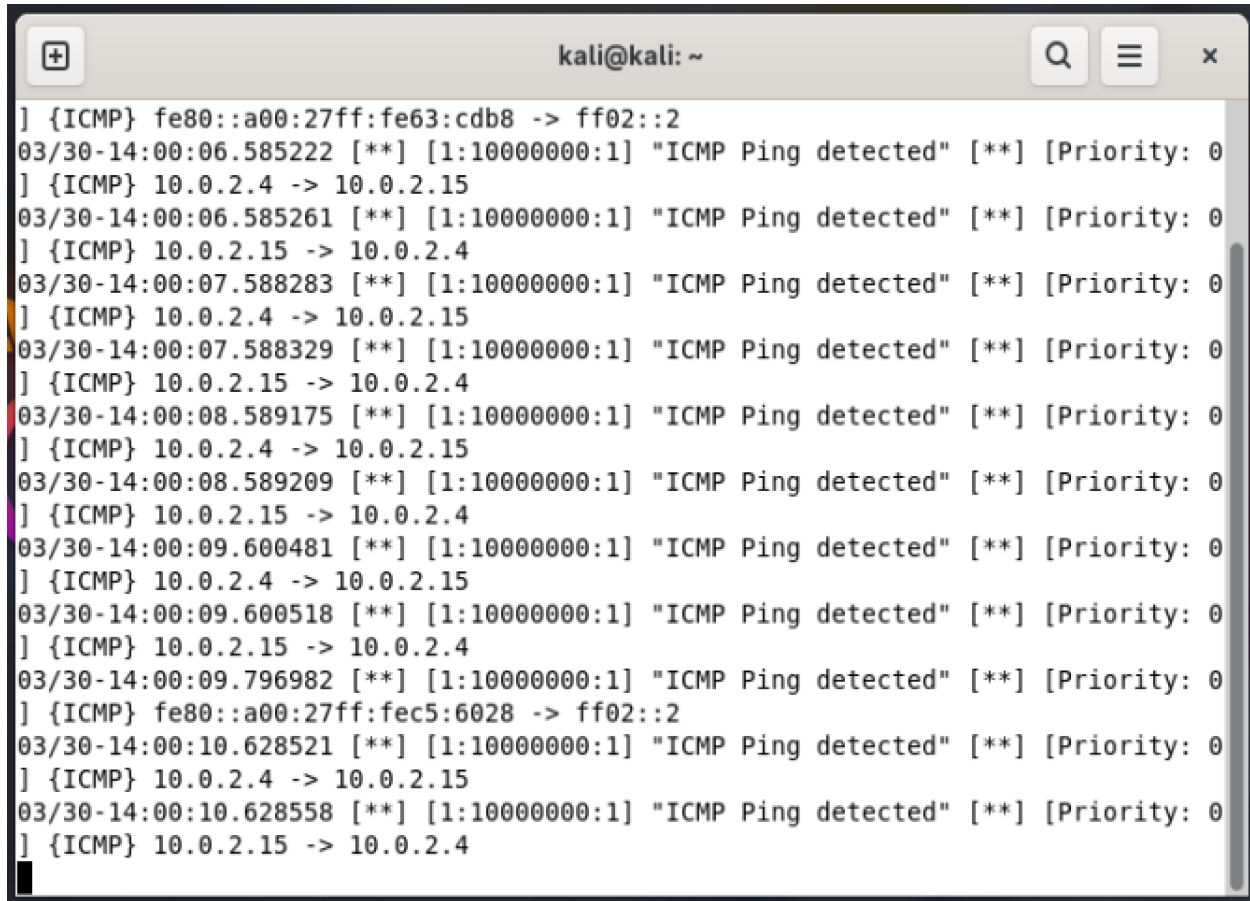
Dr. Daniel Pittman

02 April, 2026

Contents

1	Install and Configure Snort	1
2	Writing Detection Rules	2
2.1	Port Scan Detection	2
2.2	SSH Brute Force Detection	3
2.3	Custom Content Rule	4
3	Alert Analysis	5
3.1	Alert 1: SSH Brute Force Attack	6
3.2	Alert 2: SQL Injection Attempt	6
3.3	Overview:	7
4	Reflection Questions	7
4.1	IDS & Firewall:	7
4.2	False Positives:	7
4.3	The Target Lesson:	7

1 Install and Configure Snort



```
kali@kali: ~  
] {ICMP} fe80::a00:27ff:fe63:cdb8 -> ff02::2  
03/30-14:00:06.585222 [**] [1:10000000:1] "ICMP Ping detected" [**] [Priority: 0  
] {ICMP} 10.0.2.4 -> 10.0.2.15  
03/30-14:00:06.585261 [**] [1:10000000:1] "ICMP Ping detected" [**] [Priority: 0  
] {ICMP} 10.0.2.15 -> 10.0.2.4  
03/30-14:00:07.588283 [**] [1:10000000:1] "ICMP Ping detected" [**] [Priority: 0  
] {ICMP} 10.0.2.4 -> 10.0.2.15  
03/30-14:00:07.588329 [**] [1:10000000:1] "ICMP Ping detected" [**] [Priority: 0  
] {ICMP} 10.0.2.15 -> 10.0.2.4  
03/30-14:00:08.589175 [**] [1:10000000:1] "ICMP Ping detected" [**] [Priority: 0  
] {ICMP} 10.0.2.4 -> 10.0.2.15  
03/30-14:00:08.589209 [**] [1:10000000:1] "ICMP Ping detected" [**] [Priority: 0  
] {ICMP} 10.0.2.15 -> 10.0.2.4  
03/30-14:00:09.600481 [**] [1:10000000:1] "ICMP Ping detected" [**] [Priority: 0  
] {ICMP} 10.0.2.4 -> 10.0.2.15  
03/30-14:00:09.600518 [**] [1:10000000:1] "ICMP Ping detected" [**] [Priority: 0  
] {ICMP} 10.0.2.15 -> 10.0.2.4  
03/30-14:00:09.796982 [**] [1:10000000:1] "ICMP Ping detected" [**] [Priority: 0  
] {ICMP} fe80::a00:27ff:fec5:6028 -> ff02::2  
03/30-14:00:10.628521 [**] [1:10000000:1] "ICMP Ping detected" [**] [Priority: 0  
] {ICMP} 10.0.2.4 -> 10.0.2.15  
03/30-14:00:10.628558 [**] [1:10000000:1] "ICMP Ping detected" [**] [Priority: 0  
] {ICMP} 10.0.2.15 -> 10.0.2.4
```

Figure 1. Screenshot of Snort detecting ICMP pings from Ubuntu VM.

2 Writing Detection Rules

2.1 Port Scan Detection

- alert tcp any any -> \$HOME_NET any (msg:"PORT SCAN DETECTED - TCP SYN Flood"; flags:S; detection_filter:track by_src, count 20, seconds 5; sid:1000001; rev:1;)

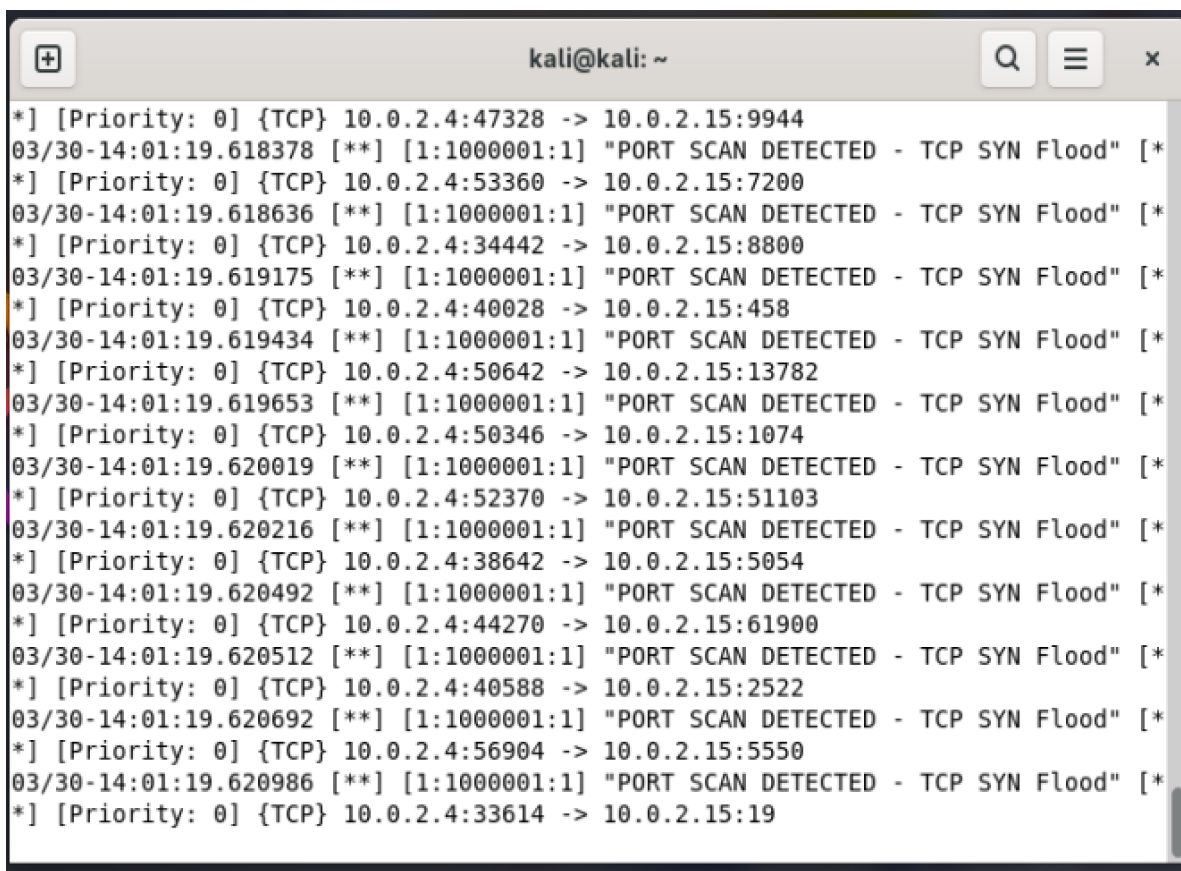
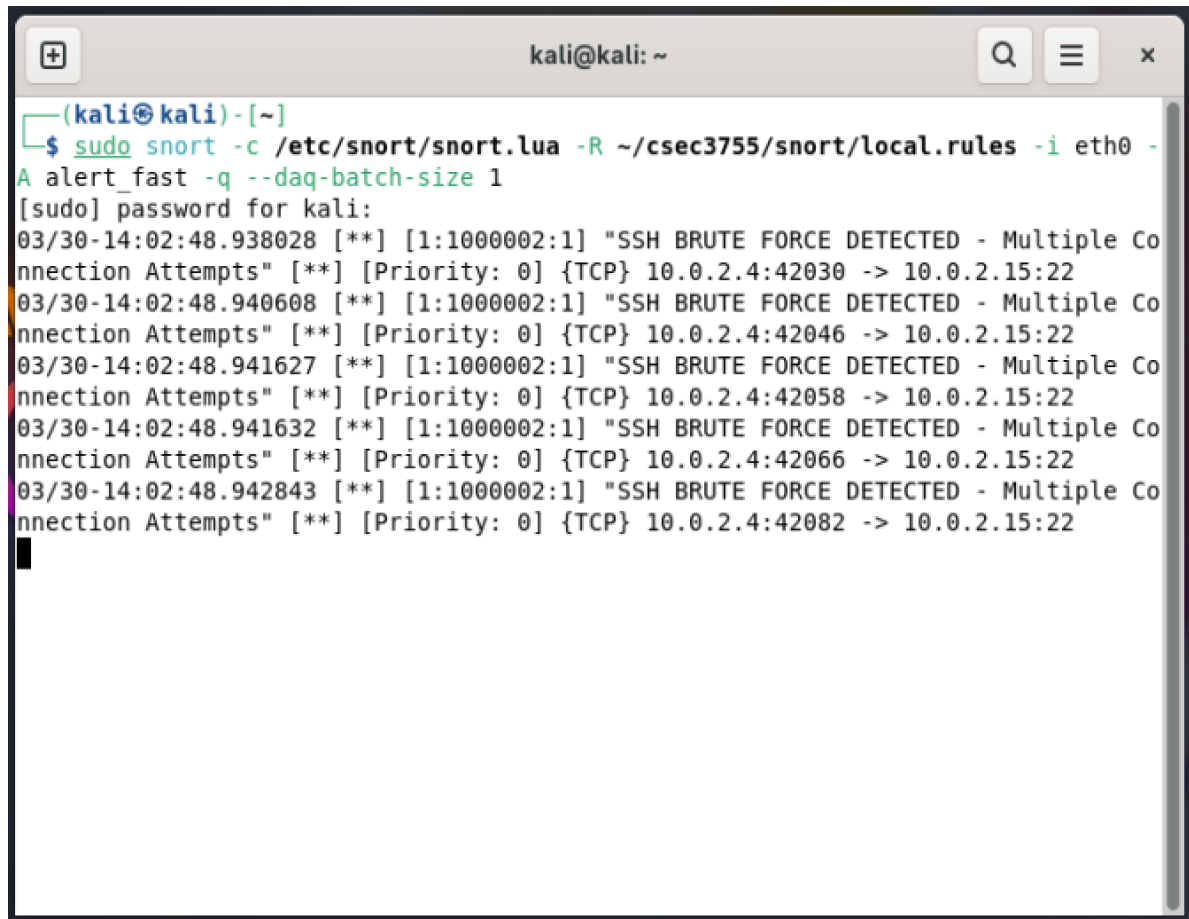


Figure 2. Snort alert for TCP SYN flood port scan.

- Setting the threshold to 20 packets in 5 seconds attempts to strike a balance between potentially having regular traffic trigger a warning when set too low and letting a clever attacker use more deliberate scans that delay allow them to limit scan rates and stay under the radar. Since a default Nmap scan blasts through 100s of ports in seconds, this threshold should easily catch suspect activity while allowing normal activity to move quietly.

2.2 SSH Brute Force Detection

- `alert tcp any any -> $HOME_NET 22 (msg:"SSH BRUTE FORCE DETECTED - Multiple Connection Attempts"; flags:S; detection_filter:track by_src, count 5, seconds 10; sid:1000002; rev:1;)`



```

kali@kali: ~
(kali@kali)-[~]
$ sudo snort -c /etc/snort/snort.lua -R ~/csec3755/snort/local.rules -i eth0 -A alert_fast -q --daq-batch-size 1
[sudo] password for kali:
03/30-14:02:48.938028 [**] [1:1000002:1] "SSH BRUTE FORCE DETECTED - Multiple Co
nnection Attempts" [**] [Priority: 0] {TCP} 10.0.2.4:42030 -> 10.0.2.15:22
03/30-14:02:48.940608 [**] [1:1000002:1] "SSH BRUTE FORCE DETECTED - Multiple Co
nnection Attempts" [**] [Priority: 0] {TCP} 10.0.2.4:42046 -> 10.0.2.15:22
03/30-14:02:48.941627 [**] [1:1000002:1] "SSH BRUTE FORCE DETECTED - Multiple Co
nnection Attempts" [**] [Priority: 0] {TCP} 10.0.2.4:42058 -> 10.0.2.15:22
03/30-14:02:48.941632 [**] [1:1000002:1] "SSH BRUTE FORCE DETECTED - Multiple Co
nnection Attempts" [**] [Priority: 0] {TCP} 10.0.2.4:42066 -> 10.0.2.15:22
03/30-14:02:48.942843 [**] [1:1000002:1] "SSH BRUTE FORCE DETECTED - Multiple Co
nnection Attempts" [**] [Priority: 0] {TCP} 10.0.2.4:42082 -> 10.0.2.15:22

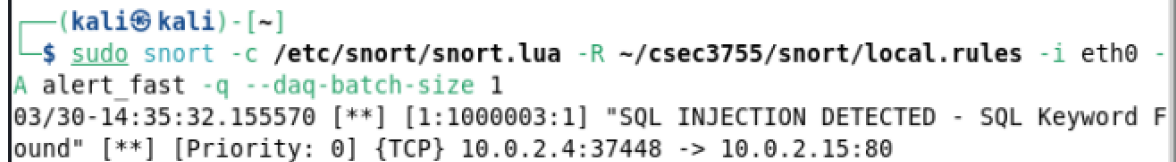
```

Figure 3. Snort alert for SSH brute force attack.

- I got to experience this first hand as my initial commands to the Kali machine would just keep me in a login loop. With or without the password, the time it took for the machine to check the password entered made it so the attempts were not picked up. It was only after I had Claude write up a quick command to simulate the attack that we saw the alert go off. This was a good first-hand show that an attacker would have to slow down to a snail's pace to go undetected.

2.3 Custom Content Rule

- alert tcp any any -> \$HOME_NET 80 (msg:"SQL INJECTION DETECTED - SQL Keyword Found"; flow:to_server,established; pcre:"/SELECT—UNION—DROP/" sid:1000003; rev:1;)



```
(kali㉿kali)-[~]
└─$ sudo snort -c /etc/snort/snort.lua -R ~/csec3755/snort/local.rules -i eth0 -A alert_fast -q --daq-batch-size 1
03/30-14:35:32.155570 [**] [1:1000003:1] "SQL INJECTION DETECTED - SQL Keyword Found" [**] [Priority: 0] {TCP} 10.0.2.4:37448 -> 10.0.2.15:80
```

-

Figure 4. Snort alert for SQL injection attempt of keywords *SELECT*, *UNION*, and *DROP*.

- This rule detects the use of common SQL keywords *SELECT*, *UNION*, and *DROP*. These keywords are often used in SQL injection attacks to retrieve data (*SELECT*), combine results (*UNION*), or delete data (*DROP*). This rule should not generate false positives for normal web traffic since these keywords are unlikely to appear in typical HTTP requests.

3 Alert Analysis

Alert Output:

```
$ sudo snort -c /etc/snort/snort.lua -R /csec3755/snort/local.rules -i eth0 -A alert_fast -q  
-daq-batch-size 1
```

sudo password for kali:

```
03/30-17:30:32.360484 ** [1:1000002:1] "SSH BRUTE FORCE DETECTED - Multiple  
Connection Attempts" ** [Priority: 0] TCP 10.0.2.4:34480 -> 10.0.2.15:22  
03/30-17:30:32.366998 ** [1:1000002:1] "SSH BRUTE FORCE DETECTED - Multiple  
Connection Attempts" ** [Priority: 0] TCP 10.0.2.4:34490 -> 10.0.2.15:22  
03/30-17:30:32.368117 ** [1:1000002:1] "SSH BRUTE FORCE DETECTED - Multiple  
Connection Attempts" ** [Priority: 0] TCP 10.0.2.4:34506 -> 10.0.2.15:22  
03/30-17:30:32.369264 ** [1:1000002:1] "SSH BRUTE FORCE DETECTED - Multiple  
Connection Attempts" ** [Priority: 0] TCP 10.0.2.4:34516 -> 10.0.2.15:22  
03/30-17:30:32.370099 ** [1:1000002:1] "SSH BRUTE FORCE DETECTED - Multiple  
Connection Attempts" ** [Priority: 0] TCP 10.0.2.4:34530 -> 10.0.2.15:22  
03/30-17:30:42.169653 ** [1:1000003:1] "SQL INJECTION DETECTED - SQL Keyword  
Found" ** [Priority: 0] TCP 10.0.2.4:34166 -> 10.0.2.15:80  
03/30-17:30:47.205335 ** [1:10000000:1] "ICMP Ping detected" ** [Priority: 0] ICMP  
fe80::a00:27ff:fe63:cdb8 -> ff02::2  
03/30-17:32:43.769580 ** [1:10000000:1] "ICMP Ping detected" ** [Priority: 0] ICMP  
fe80::a00:27ff:fe63:cdb8 -> ff02::2  
03/30-17:36:31.609834 ** [1:10000000:1] "ICMP Ping detected" ** [Priority: 0] ICMP  
fe80::a00:27ff:fe63:cdb8 -> ff02::2
```

3.1 Alert 1: SSH Brute Force Attack

Alert Output:

03/30-17:30:32.370099 [**] [1:1000002:1] "SSH BRUTE FORCE DETECTED - Multiple Connection Attempts" [**] [Priority: 0] TCP 10.0.2.4:34530 -> 10.0.2.15:22

Analysis:

SID and Revision	[1:1000002:1]
Alert Message	"SSH BRUTE FORCE DETECTED - Multiple Connection Attempts"
Source IP and Port	10.0.2.4:34530
Destination IP and Port	10.0.2.15:22
Timestamp	03/30-17:30:32.370099
Priority/Classification	0

3.2 Alert 2: SQL Injection Attempt

Alert Output:

03/30-17:30:42.169653 [**] [1:1000003:1] "SQL INJECTION DETECTED - SQL Keyword Found" [**] [Priority: 0] TCP 10.0.2.4:34166 -> 10.0.2.15:80

Analysis:

SID and Revision	[1:1000003:1]
Alert Message	"SQL INJECTION DETECTED - SQL Keyword Found"
Source IP and Port	10.0.2.4:34166
Destination IP and Port	10.0.2.15:80
Timestamp	03/30-17:30:42.169653
Priority/Classification	0

3.3 Overview:

When taking an overview of the Snort outputs, my immediate attention turns to the SSH brute force warnings. The five rapid connection attempts are textbook signs of a potential credential stuffing attack and represents a clear, active threat. If successful, the attacker would have full control of the machine. The SQL injection alerts would be my second priority, although they do present an ugent threat as even a few show that an attacker could be probing for vulnerabilities, a success meaning leaked or damaged data. Finally, I would look at the ICMP alerts, although those are just signs of normal discovery traffic. They are more of an indicator that the rule should be tightened up a little and are a good example of false positives.

4 Reflection Questions

4.1 IDS & Firewall:

4.2 False Positives:

The rule as it stands just alerts us to every ping that is detected, so we just have to turn it down a little. We can add itype:8 to match ICMP Echo Requests only and ignore router discovery and multicast traffic. Additionally, we can add detection_filter:track by_src, count 5, seconds 3 to the rule to only pay attention to multiple pings in quick succession from the same source.

4.3 The Target Lesson:

++report written with (minimal) help from Claude (Sonnet 4.6)